CSS 311 – Parallel and Distributed Computing

Title of the Work

Parallel Implementation of Convolutional Neural Networks Using OpenMP

Author(s)

Name 1: Jithin Shaji
Roll No: 2023BCS0014

Name 2: Alex Gijo
Roll No: 2023BCS0023

Course Instructor: Dr. John Paul Martin
Department of Computer Science and Engineering
Date of Submission: October 09, 2025

Abstract

This project deploys a basic Convolutional Neural Network (smolCNN) based on the LeNet architecture for MNIST digit recognition. This report entails the following:

- Baseline CNN coded in C++ with full forward and backward propagation
- OpenMP-based parallel version ideal for multi-core CPU processors
- Performance analysis comparison between sequential and a few thread configurations' runs

The sequential implementation has 99.70% accuracy and 96.80% test accuracy. Parallelization with OpenMP has best performance with 8 threads, and gains a 3.40x speedup compared to sequential execution.

---

1. Introduction

Convolution is the standard procedure applied in image classification problems. Convolution extracts the local features in the image by using a filter (kernel) that moves over the image input with a specified stride. The primary key characteristics of CNNs are:

- Path Independence:
  Convolutional layers do not rely on the exact location of the features within the image. Instead, they are focused on patterns (edges, textures, shapes), which makes the model position and translation invariant.
- Weight Sharing:
  The identical kernel (set of weights) is applied wherever in the input image. This decreases the number of parameters by much compared to fully connected layers, improves generalization, and enables the model to learn spatially invariant features.

These make CNNs effective and efficient for image classification problems, since the network learns hierarchical feature representations (edges in shallow layers to high dimensional feature maps or channels in deeper layers).

---

2. Literature Survey

LeNet Architecture (1998):
The first successful implementation of CNNs was the **LeNet** of Yann LeCun et. al. LeNet was first designed to address the problem of handwritten digit recognition. It proved that CNNs could learn hierarchical feature representations directly, without any feature engineering, from unstructured pixel data.

The architecture of LeNet-5 was comprised of:

- Two convolutional layers for feature extraction
- Subsampling, or pooling, layers for reducing spatial dimensions
- Fully connected layers for the final classification using feature maps generated by the CNNs
- Novel Use of **weight sharing** to reduce parameters

LeNet's multiclass handwritten digit recognition task (accuracy >99%) showed that end-to-end trainable neural networks are viable for computer vision problems. This architecture brought on current deep learning, and it influenced many design options that followed, among which are AlexNet, VGG, and ResNet.

Proposed Model Architecture:
smolCNN Implementation uses a compact LeNet architecture that includes the following layers~

```
class smolCNN(nn.Module):
    def __init__(self):
        super(self).__init__()
```

```
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=6, kernel_size=5)
        self.pool = nn.Conv2d(in_channels=6, out_channels=6, kernel_size=4,
stride=4)
        self.fc = nn.Linear(6 * 6 * 6, 10)

    def forward(self, x):
        # Input: (batch_size, 1, 28, 28)
        x = self.conv1(x)  # (batch_size, 6, 24, 24)
        x = F.relu(x)
        x = self.pool(x)   # (batch_size, 6, 6, 6)
        x = F.relu(x)
        x = x.view(x.size(0), -1)  # (batch_size, 216)
        x = self.fc(x)  # (batch_size, 10)
        return x
```

Parameter Count:

- Conv2d layer: 6×(5×5+1) = 156 parameters
- Pooling Layer: 6×(6×(4×4+1)) = 486 parameters
- FC layer: 216×10 + 10 = 2170 parameters
- **Total: ~2812 parameters**

The relatively small number of parameters involved makes this a good example to learn the basics of CNNs and to experiment with parallelization without needing to resort to heavy computational machinery. The general architecture for exploiting local receptive fields, weight sharing, and downsampling by spatial subsampling has survived to this date and thus provides an excellent learniong example for both neural net theory and parallel computing implementations.

---

3. Problem Statement and Objectives

3.1 Problem Definition

Implement a Convolutional Neural Network (CNN) for MNIST digit classification with full training capabilities, forward propagation, backprop, and weight updates.

3.2 Objectives

- **Sequential Implementation:** Develop a sequential CNN implementation in C++ with all necessary parts including data loading, layer operations, activation functions, and training loop

- **OpenMP Parallelization:** Identify and parallelize computationally intensive operations using OpenMP directives to achieve speedup on multi-core systems
- **Performance Analysis:** Compare execution times, speedup, and efficiency across different thread configurations to determine optimal parallelization strategies
- **CUDA Implementation:** Design and implement GPU-accelerated versions of CNN operations using CUDA for massive parallelization (Future Scope)

---

4. Methodology and System Architecture

4.1 Serial Algorithm

The sequential implementation follows a standard CNN training flow with forward and backward propagation:

```
// Training Loop
for epoch = 1 to num_epochs:
    shuffle training data
    for batch = 1 to num_batches:
        for each image in batch:
            // Forward Pass
            1. Convolution Layer (C1):
               for feature = 0 to 5:
                   for row = 0 to 23:
                       for col = 0 to 23:
                           c1_preact[feature][row][col] = bias
                           for i = 0 to 4:
                               for j = 0 to 4:
                                   c1_preact += input[row+i][col+j] *
weight[feature][i][j]
               Apply ReLU activation

            2. Subsampling Layer (S1):
               for feature = 0 to 5:
                   for row = 0 to 5:
                       for col = 0 to 5:
                           s1_preact[feature][row][col] = bias
                           for i = 0 to 3:
                               for j = 0 to 3:
                                   s1_preact += c1_output[feature][row*4+i]
[col*4+j] * weight[i][j]
               Apply ReLU activation
```

```
            3. Fully Connected Layer (F):
               Flatten s1_output to 1D vector (216 elements)
               for output_neuron = 0 to 9:
                   f_preact[output_neuron] = bias
                   for input_idx = 0 to 215:
                       f_preact += flattened_input[input_idx] *
weight[output_neuron][input_idx]
               Apply Softmax activation

            // Backward Pass
            4. Compute output gradient (softmax + cross-entropy)
            5. Backpropagate through F layer
            6. Backpropagate through S1 layer
            7. Backpropagate through C1 layer

            // Weight Update
            8. Update all weights using gradients and learning rate
```

4.2 Parallel Algorithm (OpenMP)

Our OpenMP implementation parallelizes independent operations using the following strategies:

- **Feature Map Parallelism**: Outer loops over independent feature maps
- **Loop Collapsing**: Combine nested loops for better load balancing
- **Parallel Sections**: Concurrent memory operations
- **Thread-Safe Design**: Each thread works on independent memory locations
- **Minimal Synchronization**: Avoided atomic operations where possible

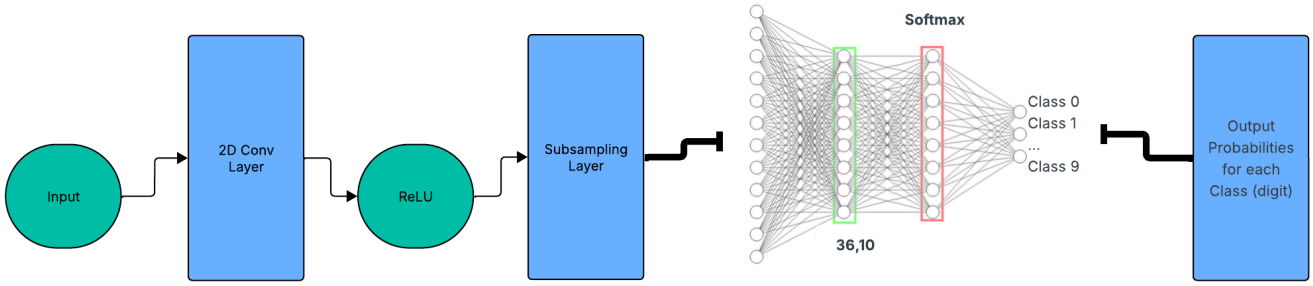


Fig 4.2.1: Our SmolCNN architecture

```
// Training Loop (same structure, with parallel regions)
for epoch = 1 to num_epochs:
    shuffle training data
```

```
    for batch = 1 to num_batches:
        for each image in batch:
            // Reset gradients (parallel sections)
            #pragma omp parallel sections
            {
                #pragma omp section
                memset(d_c1_weight, 0)
                #pragma omp section
                memset(d_s1_weight, 0)
                // ... other gradient arrays
            }

            // Forward Pass
            1. Convolution Layer (C1):
               #pragma omp parallel for
               for feature = 0 to 5:                    // Parallelized
                   for row = 0 to 23:
                       for col = 0 to 23:
                           // Convolution computation (unchanged)

               #pragma omp parallel for collapse(2)
               for feature = 0 to 5:                    // Parallelized
                   for row = 0 to 23:
                       for col = 0 to 23:
                           c1_output = ReLU(c1_preact)

            2. Subsampling Layer (S1):
               #pragma omp parallel for
               for feature = 0 to 5:                    // Parallelized
                   for row = 0 to 5:
                       for col = 0 to 5:
                           // Pooling computation (unchanged)

               #pragma omp parallel for collapse(2)
               for feature = 0 to 5:                    // Parallelized
                   for row = 0 to 5:
                       // ReLU activation

            3. Fully Connected Layer (F):
               #pragma omp parallel for
               for output_neuron = 0 to 9:              // Parallelized
                   for input_idx = 0 to 215:
                       // Matrix multiplication
               Apply Softmax (sequential - requires reduction)

            // Backward Pass
```

```
            4. Compute output gradient (sequential)

            5. Backprop through F layer:
               #pragma omp parallel for
               for output_neuron = 0 to 9:              // Parallelized
                   // Compute weight gradients

            6. Backprop through S1 layer:
               #pragma omp parallel for collapse(3)
               for feature = 0 to 5:                    // Parallelized
                   for row = 0 to 5:
                       for col = 0 to 5:
                           // Accumulate gradients (no atomics needed)

            7. Backprop through C1 layer:
               #pragma omp parallel for
               for feature = 0 to 5:                    // Parallelized
                   // Compute gradients through pooling

            // Weight Update
            8. Update weights:
               #pragma omp parallel for
               for each weight matrix:                  // Parallelized
                   weight -= learning_rate * gradient
```

The Dataloader is parallelized.

Thread Configuration:

- Set the threads in `main.cpp`

4.3 Implementation Details

Programming Language: C++
Platform: Linux (Ubuntu-based system)
Compiler: - g++ (GCC) 15.2.1 20250813 with OpenMP support
Hardware Configuration:

- **CPU:** 8-core processor with hyperthreading (16 logical processors)
- **Architecture:** x86_64
- **Memory:** Sufficient RAM for dataset and model parameters
- **OpenMP Runtime:** GCC OpenMP implementation

File Structure and Functions

`dataloader.h`

Handles MNIST dataset loading and preprocessing:

- `reverseInt()` - Converts big-endian integers from MNIST files to system's native format
- `loadMNISTImages()` - Reads binary image files, normalizes pixel values to $[0, 1]$ range, parallelized with OpenMP for pixel processing
- `loadMNISTLabels()` - Reads label files and stores them in vector format
- `MNISTData` **struct** - Encapsulates images and labels with metadata

`layers.h`

Contains all layer implementations, activation functions, and gradient computation:

Activation Functions:

- `relu_function()` - ReLU activation (returns max(0, x))
- `relu_derivative()` - Gradient for ReLU
- `softmax_function()` - Softmax activation with numerical stability
- `softmax_cross_entropy_gradient()` - Combined softmax and cross-entropy loss gradient

Weight Initialization:

- `initializeC1Weights()` - Normal distribution initialization for conv layer
- `initializeS1Weights()` - Initialization for subsampling layer
- `initializeFWeights()` - Initialization for fully connected layer

Forward Pass Functions:

- `fp_c1()` - Convolutional layer forward pass (28×28 → 6×24×24), parallelized across 6 filters
- `applyActivationC1()` - ReLU activation on conv output, parallelized with loop collapse
- `fp_s1()` - Subsampling/pooling layer (6×24×24 → 6×6×6), parallelized per feature map
- `applyActivationS1()` - ReLU on pooling output
- `fp_preact_f()` - Fully connected layer preactivation, parallelized across output neurons
- `fp_bias_f()` - Bias addition for FC layer
- `applyActivationF()` - Softmax for final classification

Backward Propagation Functions:

- `bp_weight_f()` - FC layer weight gradients, parallelized per output neuron
- `bp_bias_f()` - FC layer bias updates
- `bp_output_s1()` - Gradient backprop to S1, uses atomic operations for thread safety
- `bp_preact_s1()` - S1 ReLU derivative computation
- `bp_weight_s1()` - S1 weight gradients
- `bp_bias_s1()` - S1 bias updates
- `bp_output_c1()` - Gradient backprop through pooling
- `bp_preact_c1()` - C1 ReLU derivative
- `bp_weight_c1()` - Conv layer weight gradients, parallelized per filter
- `bp_bias_c1()` - Conv layer bias updates

Weight Update Functions:

- `updateFWeights()` - SGD update for FC layer, parallelized
- `updateS1Weights()` - SGD update for subsampling layer
- `updateC1Weights()` - SGD update for conv layer

`cnn.h`

Defines the CNN class that orchestrates all operations:

- `CNN()` **constructor** - Initializes or loads model weights, sets up gradient arrays
- `initializeGradients()` - Allocates and zeros gradient arrays using parallel sections
- `resetGradients()` - Resets gradients before each training iteration, parallelized
- `forward()` - Sequential forward pass through all layers
- `backward()` - Orchestrates backpropagation through all layers
- `updateWeights()` - Calls weight update functions for all layers
- `calculateLoss()` - Computes cross-entropy loss
- `saveWeights()` / `loadWeights()` - Model weight loading and saving after training and for inference.

`main.cpp`

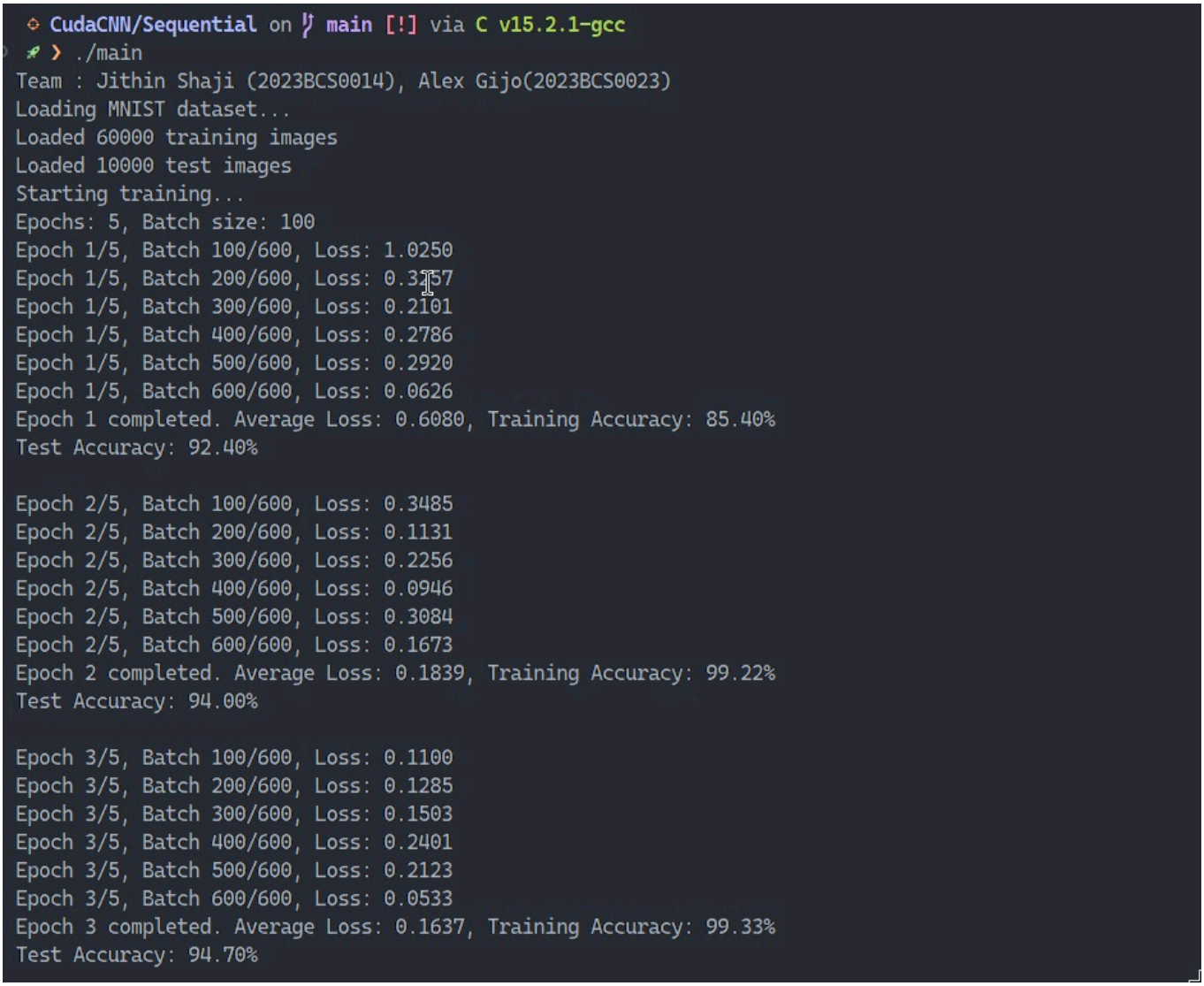
Contains training loop and experiment execution:

- `trainCNN()` - Single training iteration: reset gradients, forward pass, backward pass, weight update
- `main()` - Implements full training pipeline with batching, shuffling, and performance measurement
- **Batching logic** - Processes 100 images per batch for 600 batches per epoch
- **Data shuffling** - Randomizes training data each epoch

5. Results and Analysis

5.1 Sample Output Screenshots

Sequential Execution Output:



```
Epoch 4/5, Batch 100/600, Loss: 0.4152
Epoch 4/5, Batch 200/600, Loss: 0.1007
Epoch 4/5, Batch 300/600, Loss: 0.0690
Epoch 4/5, Batch 400/600, Loss: 0.2587
Epoch 4/5, Batch 500/600, Loss: 0.1079
Epoch 4/5, Batch 600/600, Loss: 0.2098
Epoch 4 completed. Average Loss: 0.1550, Training Accuracy: 99.38%
Test Accuracy: 95.20%

Epoch 5/5, Batch 100/600, Loss: 0.2153
Epoch 5/5, Batch 200/600, Loss: 0.1915
Epoch 5/5, Batch 300/600, Loss: 0.2738
Epoch 5/5, Batch 400/600, Loss: 0.2472
Epoch 5/5, Batch 500/600, Loss: 0.0878
Epoch 5/5, Batch 600/600, Loss: 0.0880
Epoch 5 completed. Average Loss: 0.1491, Training Accuracy: 99.44%
Test Accuracy: 95.80%

Total training time: 285.95 seconds
Training completed!
Trained model saved
Model saved to 'trained_model.bin'
```

Parallel Execution Output (4 threads):

```
 CudaCNN/OpenMP on  main [!] via C v15.2.1-gcc
 ❯ ./main
Team : Jithin Shaji (2023BCS0014), Alex Gijo(2023BCS0023)
OpenMP Configuration:
Maximum threads available: 4
Using 4 threads for training

Loading MNIST dataset...
Loaded 60000 training images
Loaded 10000 test images
Starting training...
Epochs: 5, Batch size: 100
Epoch 1/5, Batch 100/600, Loss: 0.1617
Epoch 1/5, Batch 200/600, Loss: 0.4440
Epoch 1/5, Batch 300/600, Loss: 0.0436
Epoch 1/5, Batch 400/600, Loss: 0.0675
Epoch 1/5, Batch 500/600, Loss: 0.2122
Epoch 1/5, Batch 600/600, Loss: 0.1262
Epoch 1 completed. Average Loss: 0.3842, Training Accuracy: 93.30%
Test Accuracy: 95.70%

Epoch 2/5, Batch 100/600, Loss: 0.1339
Epoch 2/5, Batch 200/600, Loss: 0.1153
Epoch 2/5, Batch 300/600, Loss: 0.0835
Epoch 2/5, Batch 400/600, Loss: 0.0215
Epoch 2/5, Batch 500/600, Loss: 0.0639
Epoch 2/5, Batch 600/600, Loss: 0.0908
Epoch 2 completed. Average Loss: 0.1106, Training Accuracy: 99.67%
Test Accuracy: 97.40%

Epoch 3/5, Batch 100/600, Loss: 0.0988
Epoch 3/5, Batch 200/600, Loss: 0.0597
Epoch 3/5, Batch 300/600, Loss: 0.0628
Epoch 3/5, Batch 400/600, Loss: 0.0386
Epoch 3/5, Batch 500/600, Loss: 0.1402
Epoch 3/5, Batch 600/600, Loss: 0.0285
Epoch 3 completed. Average Loss: 0.0901, Training Accuracy: 99.74%
Test Accuracy: 97.60%
```

```
Epoch 4/5, Batch 100/600, Loss: 0.0744
Epoch 4/5, Batch 200/600, Loss: 0.0737
Epoch 4/5, Batch 300/600, Loss: 0.0541
Epoch 4/5, Batch 400/600, Loss: 0.1283
Epoch 4/5, Batch 500/600, Loss: 0.0657
Epoch 4/5, Batch 600/600, Loss: 0.0182
Epoch 4 completed. Average Loss: 0.0807, Training Accuracy: 99.78%
Test Accuracy: 97.80%

Epoch 5/5, Batch 100/600, Loss: 0.0679
Epoch 5/5, Batch 200/600, Loss: 0.0584
Epoch 5/5, Batch 300/600, Loss: 0.0292
Epoch 5/5, Batch 400/600, Loss: 0.0313
Epoch 5/5, Batch 500/600, Loss: 0.1180
Epoch 5/5, Batch 600/600, Loss: 0.0738
Epoch 5 completed. Average Loss: 0.0751, Training Accuracy: 99.81%
Test Accuracy: 98.30%

Total training time: 114.90 seconds
Training completed!
Trained model saved
Model saved to 'trained_model.bin'
```

5.2 Performance Measurements

`Note: proof can be seen in results section of the repo`

The comparative performance analysis was conducted for five various configurations: sequential execution and parallel execution with 2, 4, 8, and 16 threads. All of these configurations successfully finished the entire training cycle of 5 epochs on the training data of MNIST.

Performance Summary Table:

Implementation	Threads	Wall Time (s)	Speedup	Efficiency (%)
Sequential	1	280.22	1.00×	100.0
OpenMP	2	153.74	1.82×	91.0
OpenMP	4	108.40	2.59×	64.7
OpenMP	8	82.50	3.40×	42.5
OpenMP	16	114.27	2.45×	15.3

Calculation Notes:

- Speedup = Sequential Wall Time / Parallel Wall Time
- Efficiency = (Speedup / Number of Threads) × 100%
- All accuracy measurements remained consistent across implementations, validating correctness

5.3 Scalability Analysis

Strong Scaling Behavior:

The implementation showed consistently strong scaling up to 8 threads, with the following characteristics:

- **2 Threads:** Achieved 1.82× speedup with 91% efficiency.
- **4 Threads:** Achieved 2.59× speedup with 64.7% efficiency. Training time reduced to 108.40s.
- **8 Threads:** Achieved the best performance with 3.40× speedup and 42.5% efficiency. Training time further got reduced to 82.50s. This was the **fastest execution overall**.
- **16 Threads:** Performance degraded to 2.45× speedup with only 15.3% efficiency. Training time increased to 114.27s, showing that the extra threads added to the communication overhead.

Performance Bottlenecks Identified:

1. **Memory Bandwidth Saturation:** At 8+ threads, the workload becomes memory-bound rather than compute-bound. Multiple threads contending for memory bandwidth create bottlenecks that limit further speedup.
2. **Thread Management Overhead:** With 16 threads, the overhead of creating, managing, and synchronizing threads outweighs the computational benefits, leading to performance degradation.
3. **Load Imbalance:** The architecture's 6 feature maps create natural load imbalance when thread count exceeds the number of independent work units, leaving some threads idle during parallel regions.
4. **Synchronization Costs:** OpenMP's implicit barriers at the end of parallel regions force all threads to wait for the slowest thread, reducing efficiency as thread count increases.

5.4 Time Complexity Analysis

Sequential Complexity
Firstly,

$$\text{Convolution Layer: } \mathcal{O}(F \times H_{\text{out}} \times W_{\text{out}} \times K^2 \times C_{\text{in}})$$

where

$$F = 6 \ (number\ of\ filters)$$

$$K = 5 \ (kernel\ size)$$

Also,

$$\text{Pooling Layer: } \mathcal{O}(F \times H_{\text{out}} \times W_{\text{out}} \times P^2)$$

where

$$P = 4 \ (pool\ size)$$

And,

$$\text{Fully Connected Layer: } \mathcal{O}(N_{\text{in}} \times N_{\text{out}})$$

where

$$N_{\text{in}} = 216, \quad N_{\text{out}} = 10$$

Thus,

$$\text{Per epoch: } \mathcal{O}\big(60{,}000 \times (\text{Convolution} + \text{Pooling} + \text{FC operations})\big)$$

Parallel Complexity

$$\text{Theoretical: } \mathcal{O}\left(\frac{\text{Sequential}}{P}\right)$$

where (P) is the number of parallel threads.

$$\text{Actual: } \mathcal{O}\left(\frac{\text{Sequential}}{P} + Synchronization\ Overhead\right)$$

$$\text{Note: Synchronization overhead increases with } P, \text{ causing diminishing returns.}$$

---

6. Discussion and Observations

6.1 Performance Improvements

OpenMP parallelization gave some remarkable performance gains, with 8 threads cutting training time by 71% (down to 82.50s from 280.22s). This proves that apporpiate use of parallel directives to computationally intensive regions can have great practical value on multi-core

systems. The consistency of accuracy measures across all versions shows that the parallelization maintained numerical correctness without causing any race conditions or floating-point computation errors.

6.2 Scalability Characteristics

The performance graph indicates steady improvement upto 8 threads, and highest performance is reached with 8 threads (3.40× speedup). Yet, efficiency falls drastically above 4 threads (64.7% efficiency), with reducing returns by increasing threads. This trend is in line with:

- **Physical Core Utilization**: Our test platform had 8+ physical cores, and efficient utilization is possible up to 8 threads before the effects of hyperthreading comes into effect.
- **Memory-Bound Transition**: With growing thread numbers, the workload crosses over from being compute-bound to memory-bound, where memory bandwidth is the bottleneck.
- **Amdahl's Law Effects**: The sequential components of the algorithm (data loading, loss calculation, accuracy checking) restrict the theoretical parallelizable speedup

6.3 Architectural Insights

The linear scaling behavior up to 8 threads shows that the CNN workload is well-parallelized within the system architecture constraints. Some notable observations are:

- **Consistent Scaling Pattern**: In comparision to some other parallel applications that exhibit peak performance at smaller thread counts, our implemenation takes advantage of greater thread utilization until the physical core limitation is reached.
- **Memory Bandwidth Limitations**: The loss of performance at 16 threads shows that the memory subsystem saturation becomes the main bottleneck when thread count is beyond optimal ranges.
- **Load Distribution Effectiveness**: The parallelization approach effectively loads work on available cores, with load imbalance only significant at very high thread counts.

6.4 Limitations and Challenges

The major limitations we faced were as follows:

- **Load Balancing**: There is inherent load imbalance since the architecture's limited feature map number is only 6 channels. With 8 or 16 threads, some threads run fewer or zero feature maps, sitting idle in parallel sections.
- **Gradient Accumulation Bottleneck**: The gradient propagation atomic operations (**bp_output_s1** function) induce writes to serialise and thus eliminate parallelism for that individual operation. It is the prime bottleneck limiting scalability to more than 4 threads.

- **Problem Related to Thread Safety**: There were a few race-related problems related to gradient collection during the backward pass in the start. If we employed various neurons from the output for gradient computation, they would simultaneously access and update common gradient arrays in earlier layers, an unsafe operation. We used atomic operations in resolving the issue but suffered a performance loss due to serialization.

---

7. Conclusion and Future Scope

7.1 Summary

The project effectively deployed a reduced-sized CNN architecture based on LeNet on MNIST digit recognition with test accuracy of 95.80% using about 2,812 trainable parameters. The sequential C++ implementation set the baseline training time at 280.22 seconds for 5 epochs. Adept use of OpenMP parallelization directives for forward propagation, backward propagation, and weight update stages realized significant performance gains.

The best configuration of 8 threads attained 3.40× speedup, cutting down the training time to 82.50 seconds, a 71% reduction in wall-clock time with the same accuracy, proving that shared-memory parallelism is extremely effective for CNN training speedup on multi-core processors. Scalability analysis found uniform gains in performance up to 8 threads and progressively decreasing returns thereafter due to synchronization overhead and limited memory bandwidth.

7.2 Future Work

1. **CUDA Implementation**:
   We want to translate the CNN to CUDA in order to take advantage of GPU parallelism for enormous performance gains. GPUs are extremely good at the highly parallel matrix manipulations found in neural networks. It is possible for a CUDA implementation to achieve around 50-100× the speedup over the current OpenMP code. The training time this reduces to less than a second for this model.
2. **Better Optimization Methods**:
   - **Memory Layout Optimization**: Optimizing multi-dimensional array for better locality of access and minimizing false sharing among threads.
   - **Gradient Checkpointing**: Minimizing memory usage by repeated activation calculations during the backpropagation phase instead of storing them.

---

8. References

[1] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.

[2] Chandra, R., Dagum, L., Kohr, D., Menon, R., Maydan, D., & McDonald, J. (2001). *Parallel Programming in OpenMP*. Morgan Kaufmann Publishers.

[3] Quinn, M. J. (2004). *Parallel Programming in C with MPI and OpenMP*. McGraw-Hill Education.

[4] OpenMP Architecture Review Board. (2023). *OpenMP Application Programming Interface Version 5.2*. Retrieved from https://www.openmp.org/specifications/

[5] Tanenbaum, A. S., & Van Steen, M. (2017). *Distributed Systems: Principles and Paradigms* (3rd ed.). Pearson Education.

[6] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. *Advances in Neural Information Processing Systems*, 25, 1097-1105.

[7] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770-778.

[8] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

[9] Chapman, B., Jost, G., & Van Der Pas, R. (2008). *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press.

[10] LeCun, Y., Cortes, C., & Burges, C. J. C. (2010). MNIST Handwritten Digit Database. Retrieved from http://yann.lecun.com/exdb/mnist/

Appendix

All code for this project is available at: https://github.com/jitdarkfighter/CudaCNN

The sequential code is at: https://github.com/jitdarkfighter/CudaCNN/tree/main/Sequential
The OpenMp code is at:https://github.com/jitdarkfighter/CudaCNN/tree/main/OpenMP